Flink SQL reports

Seven reports that read two streams of isotope metadata — the **produce side** (the `x-isotope-*` headers stamped on every event topic record by `IsotopeProducerInterceptor`) and the **consume side** (the value-less marker records on `iso_consume_events` emitted by `IsotopeContext.recordConsume`) — and surface what's flowing where, how fast, and how reliably. Each report runs as a long-lived `INSERT INTO <report>_1m SELECT …` streaming job. The aggregation logic is identical across runtimes; only the source/sink DDL and function-registration glue differ.

The headline addition is the `bipartite_topology` report: it unions the produce and consume views to render the pipeline as a literal **bipartite graph** from graph theory — services in one vertex set, topics in the other, edges crossing between them in both directions. See root README §1.0 "Background — why 'bipartite'?" for the full motivation.

Runtime	Reports	Sink format	Where the SQL lives
Confluent Platform Flink (cp-flink 2.1.2 session cluster on Minikube)	7 (latency, topology, bipartite-topology, hop-distribution, coverage, stuck-trace, latency-percentiles)	`avro-confluent` (SR-framed Avro)	scripts/flink/sql/cp/ — applied by scripts/deploy-cp-flink-reports.sh
Confluent Cloud for Apache Flink (CCAF)	6 (no percentiles — UDAFs disallowed)	`proto-registry` (SR-framed Protobuf)	Inlined as `confluent_flink_statement` resources in terraform/setup-confluent-flink.tf — applied by scripts/deploy-cc-flink-reports.sh

Control Center deserializes both sink formats natively.

What each report computes

Every report aggregates over a `TUMBLE(event_time, INTERVAL '1' MINUTE)` window. The "Source view" column names the typed view (or views) the report reads from; the views in turn filter the single `isotope_raw` source table by the presence/absence of the `x-isotope-consumer-service` header.

Report	Source view	What it computes	Runtimes
`latency`	`isotope`	avg / min / max end-to-end latency by `origin_service × current_topic`	CP, CCAF
`topology`	`isotope`	produce-edge counts per `(producer_service, topic)` per minute	CP, CCAF

Report	Source view	What it computes	Runtimes
`bipartite_topology`	`isotope` ∪ `consume_events`	full bipartite graph: produce edges + consume edges per minute	CP, CCAF
`hop_distribution`	`isotope`	record counts bucketed by `hop_count` per topic per minute	CP, CCAF
`coverage`	`isotope`	distinct traces per topic per minute	CP, CCAF
`stuck_trace`	`isotope`	alerts (via `STUCK_TRACE_PTF`) for traces idle ≥60s of event time	CP, CCAF
`latency_percentiles`	`isotope`	p50 / p95 / p99 (via `LATENCY_PERCENTILES` UDAF, T-Digest)	CP only

Format-by-runtime, not by domain

Two asymmetries shape the sink format picture, and both are platform-level — not project preferences:

- **cp-flink ships SR-Avro but not SR-Protobuf.** Apache Flink open-source publishes `flink-sql-avro-confluent-registry`; there is no SR-integrated Protobuf equivalent. We could hand-write one (~150 lines wrapping `flink-protobuf` with magic-byte framing
  - SR client) but Avro already gives us Control Center decoding with zero custom code.
- **CCAF rejects all user-defined aggregate functions** with `aggregate functions are not supported`, regardless of accumulator shape. The `LATENCY_PERCENTILES` T-Digest UDAF therefore registers on CP only. The Java class still ships in the JAR — it'll register on CCAF the day Confluent lifts the restriction. The other JAR-backed function, `STUCK_TRACE_PTF` (a ProcessTableFunction, not a UDAF), works on both runtimes.

The demo *event* topics (`iso_start`, `iso_mid`, `iso_final`) ride Protobuf+SR via the Java app's `DemoEvent` schema on both runtimes — those are written by the Kafka producer client, not by Flink, so the SR-Protobuf gap doesn't apply.

Layout

```
scripts/flink/sql/cp/                    CP Flink — session-cluster SQL
  00_source_table.fql                    CREATE TABLE with 'connector' =
'kafka' (reads iso_.*)
  01_register_functions.fql              CREATE FUNCTION … USING JAR
'file:///opt/flink/lib/isotope-flink-udf.jar'
  05_isotope_view.fql                    Typed view; decodes x-isotope-*
header scalars (produces only)
  06_consume_events_view.fql             Typed view of iso_consume_events
markers (consume edges)
  05_report_sinks.fql                    CREATE TABLE for each
isotope_report_*_1m Kafka sink (avro-confluent)
  10_latency_report.fql                  INSERT INTO: avg/min/max latency by
origin × topic
  20_topology_report.fql                 INSERT INTO: produce-edge counts
```

```
per minute
  25_bipartite_topology_report.fql      INSERT INTO: produce ∪ consume
edges (bipartite graph) per minute
  30_hop_distribution.fql               INSERT INTO: hop-count buckets per
topic per minute
  40_coverage_report.fql                INSERT INTO: distinct traces per
topic per minute
  60_stuck_trace_report.fql             INSERT INTO: stuck-trace alerts via
STUCK_TRACE_PTF
  70_latency_percentiles_report.fql     INSERT INTO: p50/p95/p99 via
LATENCY_PERCENTILES UDAF (CP only)
  99_teardown.fql                       DROP TABLE/VIEW/FUNCTION (companion
to flink-reports-down)
```

CCAF runs the same six non-percentile reports, but the SQL lives inline as `confluent_flink_statement` resources in terraform/setup-confluent-flink.tf — 20 statements: ALTER (×4) + raw view + typed produce view + typed consume view + 6 sinks + `STUCK_TRACE_PTF` registration + 6 INSERTs. The JAR is uploaded as a `confluent_flink_artifact` and referenced via `USING JAR 'confluent-artifact://<id>'`.

Wire-format detail (CP only)

Window columns ride as `BIGINT` epoch millis on the wire, not `TIMESTAMP_LTZ`. Flink 2.1.2's `avro-confluent` schema-derivation path raises `UnsupportedOperationException: Unsupported to derive Schema for type: TIMESTAMP_LTZ(3)` for that type. We cast in the INSERT (`UNIX_TIMESTAMP(CAST(window_start AS STRING)) * 1000`) so the on-wire schema is plain Avro `long`. Consumers rehydrate via `TO_TIMESTAMP_LTZ(window_start, 3)`. The CCAF Protobuf sinks don't hit this — `proto-registry` handles `TIMESTAMP_LTZ` directly.

UDF/PTF JAR

The single shadow JAR `ptf/build/libs/isotope-flink-udf.jar` (produced by `./gradlew :ptf:shadowJar`) carries both JAR-backed functions under the `ai.signalroom.kafka.isotope.flink` package:

- `LatencyPercentilesUDAF` — T-Digest accumulator → p50/p95/p99 (registered on CP only).
- `StuckTracePTF` — per-trace state + event-time timer that emits alerts for traces idle ≥60s (registered on both CP and CCAF).

The JAR ships unchanged to both runtimes; only the `CREATE FUNCTION … USING JAR …` clause differs (`file://` path on CP, `confluent-artifact://` reference on CCAF).

Operations

CP Flink (Minikube):

```
make flink-up           # cert-manager → CFK Flink Operator → CMF → session
cluster
make flink-reports-up   # build PTF JAR, copy to JM, create sink topics,
```

```
submit 7 INSERT INTO jobs
make flink-sql          # interactive SQL Client (auto-loads sink DDL so
SELECT * works)
make flink-reports-down # cancel jobs, drop tables, delete sink topics + SR
subjects
make flink-down         # tear down cluster + operator + cert-manager
```

CCAF (Confluent Cloud, Terraform-driven):

```
make cc-flink-reports-up   CONFLUENT_API_KEY=... CONFLUENT_API_SECRET=...
                           # terraform apply: env + cluster + topics +
compute pool + artifact + 20 statements
                           # also regenerates terraform/terraform.png via
`terraform graph | dot`
source scripts/cc-cli-env.sh          # exports BOOTSTRAP / SR_URL /
KAFKA_KEY / KAFKA_SECRET / SR_KEY / SR_SECRET / JAAS
scripts/cc-app-run.sh send iso_start svc-A 'hello'   # drives traffic with
the SASL config
make cc-flink-reports-down CONFLUENT_API_KEY=... CONFLUENT_API_SECRET=...
                           # terraform destroy: deletes the environment and
everything in it
```

See the root README §4.5 for the full CCAF walkthrough, including the multi-window sustained-traffic pattern required to see tumbling-window aggregates emit.